

Seven screens, and the loop that keeps them good

All seven ecosystem screens are built, wired and reachable, on two new server routes written for them. More important than the number: there is now a **way of working** that catches design and logic faults before a customer meets them — and it has already caught five.

Where the seven stand

These are the surfaces Micromart's core banking does not provide. Everything else in the app — balances, ledger, statements, applications — is theirs and unchanged.

SCREEN	ROUTE	READS	STATE
Statement crunch incl. the Safaricom theatre	/crunch	/api/portal/recrunch	BUILT
Pay by M-PESA	/pay	/api/portal/pay	BUILT
Auto-repay (Ratiba)	/auto-repay	/api/portal/standing-order	BUILT
Why this decision	/why	/api/portal/decision	BUILT
Your limit (the ladder)	/limit	/api/portal/ladder	BUILT
Exposure report CRB + Interchange	/credit-file	/api/portal/exposure	BUILT
Consent	/permissions	/api/portal/consent	BUILT

It never spends the lender's money. A live Metropol pull is billed per call and the tariff scales with scrutiny — an endpoint a customer can refresh at will is a way to hand a lender a surprise invoice, and at scale a denial-of-wallet. The route reads the last stored report from `KycCheck kind=CRB` and never calls `runCrbCheck()`. The spend decision stays with the lender, in their own console.

Consent is checked, not assumed. `crbCheck` must be granted on the borrower's latest record before a stored file is read back. Serving a report the customer has since withdrawn permission for is exactly what the withdrawal was meant to stop, and "we already had it" is not a defence.

The Interchange half is reported as not connected, deliberately. A federated query is not a fetch: the subject must be tokenised through the OPRF so no raw national ID crosses a boundary, a `consent_ref` issued against that token, and only then can the broker fan out over Ed25519-signed requests returning bands and counts — never a name, never an amount. That plumbing is Sprint 3. Saying so in one line beats an empty section that reads like the customer owes nothing anywhere.

The loop

Every one of these screens sits behind an SMS code, a national ID and — for the crunch — a real M-PESA payment. Left alone, the only way to look at one is to pay for it, on a real handset, with a real statement. Nobody runs that loop twenty times while getting the spacing right, so screens do not get looked at. This is the replacement.

1 Write the fixture first, and make it unflattering

Shaped exactly like the server's response, in

`src/pages/eco/Preview.jsx`. Round numbers and three short factors make every layout look fine. The crunch fixture carries a betting factor, a 46-character merchant string, two negative reasons and a six-item list — because those are the shapes that break a design.

2 Build against the harness, not the live API

`/preview` for the component gallery, `/preview/screen?s=pay` for a real screen, `/preview/crunch?stage=factors` to jump straight into any stage of the theatre. No session, no network, no payment. Mounted only when `DEV` or `VITE_PREVIEW=1`; a production build removes the routes entirely.

3

Review on the device wall before calling it done

`tools/devicewall.html` renders the same route in four real browsing contexts — 360, 414, 768, 1024 — side by side. Use it instead of `headless --window-size`, which lays out at a default width and crops the screenshot to the size you asked for. That discrepancy makes a sound layout look broken and a broken one look sound.

4

Wire the real route, then walk the degraded states

Session expired, lender not configured, service unreachable, nothing on file. All four are reachable in the live app and all four have their own words. A single "something went wrong" across all of them is what makes a working product feel broken when it is merely unconfigured.

THE LOOP HAS ALREADY PAID FOR ITSELF

Four faults caught in the harness, none of which would have surfaced in normal use until a customer hit them:

The credit dial rendered “-11 out of 850”. A rAF timestamp can predate the `performance.now()` captured immediately before it, so the eased progress went negative and the arc drew backwards.

The same dial could silently show 0. If rAF is throttled — a backgrounded tab, a phone that locked while the customer read the SMS — the counter never left its initial value and a customer would be shown a zero credit score. There is no worse failure on that screen. It now settles on a timer regardless.

The login panel's background was an HTML file. `background-image-8.jpg` is 111KB of `<!doctype html>`, and Vercel's SPA rewrite answered it `200 image/jpeg`, so it looked fine in a network tab while rendering as a flat brown rectangle.

An invalid enum in the exposure route. The open-loan count filtered on `ARREARS` and `DEFAULTED`. Neither is a member of `LoanStatus` — Prisma validates enums at query time, so the whole screen would have thrown on first load. Caught by reading the schema rather than assuming it; the typechecker cannot see inside a string literal in a `where`.

And one false alarm worth recording: a reported mobile overflow at 360px turned out to be the screenshot harness cropping a 500px layout. The device wall exists because of that hour.

Definition of done

A screen is not done when it renders. It is done when every one of these is true. This list is the review, and it is short enough to actually run.

States

- ☐ Loading, empty, error, lender-not-configured and session-expired all render, each with its own wording
- ☐ Every failure path offers the customer a next action, not just a description
- ☐ Nothing shows a spinner that cannot end

Truthfulness

- ☐ No figure on screen that the server did not send — placeholders are visibly placeholders and never survive the response
- ☐ Money always carries two decimals and its currency
- ☐ Nothing promises an outcome the system cannot grant

Responsive

- ☐ Reviewed on the device wall at all four widths
- ☐ No horizontal scroll at 360; wide content scrolls inside its own container
- ☐ Flex rows that can overflow carry `min-width: 0`

Interaction

- ☐ Every control has a visible keyboard focus state
- ☐ Tap targets at least 44px
- ☐ `prefers-reduced-motion` honoured — animation is enhancement, never the only path to a value

Copy

- ☐ No “unfortunately”, no apologies, no blame
- ☐ Errors say what happened and what to do next
- ☐ A control names exactly what it will do

Sprints

Sprint 1 — Foundation and the five

DONE

this session

The design layer (`eco.css` , `Surface.jsx`), the shared identity gate, the Safaricom crunch theatre, five screens wired into the sidebar, the preview harness and the device wall.

Sprint 2 — The last two screens

DONE

this session

- 2.1 `/api/portal/exposure` — reads the stored bureau file under the borrower's own consent. Never triggers a paid pull.
- 2.2 `/api/portal/consent` — GET the catalogue, grants and full history; POST to change one. Append-only, so a withdrawal never destroys the evidence of what was agreed on the day a check ran.
- 2.3 `/credit-file` — score, bureau detail, negative listings, and an honest line for each of the three ways a section can be empty.
- 2.4 `/permissions` — a toggle per grant, withdrawal as easy as granting, the consequence stated before the toggle rather than after, and the customer's own copy of the audit trail.
- 2.5 Fixture layer — `fixtures.js` swaps `window.fetch` so screens run their real loading, error and success paths against canned responses. Zero references survive a production build.

Sprint 3 — Federation, and the flows already served

NEXT

endpoints mostly exist

- 3.0 **Connect the Interchange.** OPRF tokenisation of the subject, a `consent_ref` issued on the `PWA` channel — which the exchange already recognises — then the brokered query. Turns the one honest "not switched on" line on the credit-file screen into a real band. The largest item here.
- 3.1 KYC wizard — `/api/portal/kyc`, five steps, confidence gates. Reuse Micromart's live `vision/ocr/id` rather than adding a vendor.
- 3.2 Guarantor — `/api/portal/guarantee/[id]`. Consent and decline both first-class.
- 3.3 Offer acceptance — `/api/portal/offer/[id]`.

Sprint 4 — Make it fast and make it correct

before go-live

- 4.1 **Bundle.** 754kB / 200kB gzipped today, on a phone, in Kenya, on mobile data. Route-split the eco screens and the theatre.
- 4.2 **Assets.** `safaricom-25.gif` is 890kB and renders at 80px wide; `mpesa-logo.png` is 583kB. Both want re-encoding.
- 4.3 Accessibility pass — focus order, labels, contrast on the tenant accent for every lender, not only Micromart.
- 4.4 Confirm `PAYMENTS_ENTITY_ID` with Micromart. It is `7`; entity `7` 404s on their config endpoint, so it is a different id space or it is wrong. A wrong value means no STK prompt reaches the handset.

What is in the box

PATH	WHAT IT IS
<code>components/eco/eco.css</code>	Tokens, the white-wave ground, cards, rows, chips, buttons, states, motion
<code>components/eco/Surface.jsx</code>	Screen, Card, Hero, Row, Chip, Button, State, Loading, Icon, money, when
<code>components/eco/Identity.jsx</code>	The shared gate — SMS code, then national ID. One verification, seven screens
<code>components/eco/CrunchTheatre.jsx</code>	The Safaricom sequence, ported off framer-motion to keyframes
<code>lib/ecosystem.js</code>	Same-origin client for <code>/api/portal/*</code> — every ecosystem call

PATH	WHAT IT IS
lib/micromart.js	Micromart's own rails: bearer + X-New-Token refresh, SMS, STK, OCR, FCM
lib/tenant.js	Lender resolution and theming — one entity id, one palette, every lender
pages/eco/Preview.jsx	The harness — gallery, screen mode, theatre stage-jumping
pages/eco/fixtures.js	The fetch shim and its deliberately unflattering fixtures
api/portal/exposure/route.ts	Stored bureau file, consent-gated, never a paid pull
api/portal/consent/route.ts	Grants, history and change — append-only
tools/devicewall.html	Four real viewports, side by side